

# Package Operations

Section: 2 of 11 | Subtopic: Package Operations | Duration: Part of 4 – 5 hours | [Theory](#) [Practical](#)

Topics covered: Local vs Global installs · Semantic Versioning ( `^`, `~`, `exact` ) · Updating packages · Security auditing

## Learning Objectives — This Subtopic

- Understand the difference between local and global package installations and when to use each.
- Read and write semantic version ranges confidently — `^`, `~`, and exact versions.
- Identify outdated packages and update them safely using `npm outdated` and `npm update`.
- Run a security audit with `npm audit` and interpret its output to fix vulnerabilities.

## 1. Local vs Global Installs

Every package you install with NPM has to go somewhere on your computer. NPM gives you two choices: **local** or **global**. Understanding the difference will prevent a very common class of beginner errors.

### 1.1 Local Installation (Default)

When you run `npm install <package>` without any flags, the package is installed *locally* — meaning inside the `node_modules` folder of your current project directory. This is the default and is what you should use for almost everything.

```
# Installs into ./node_modules – local to this project only
npm install express
```

The package is recorded in `package.json` under `"dependencies"` and is only accessible from within this project. A different project on your machine cannot use it.

## 1.2 Global Installation ( `-g` flag)

When you add the `-g` flag, NPM installs the package into a single *system-wide* location, making its command available from any folder in your terminal — regardless of which project you are in.

```
# Installs globally – available from anywhere in the terminal
npm install -g nodemon
```

### Analogy

Imagine you are building a house (your project). Local installation is like buying a specific tool (a saw) for *this house only* — it lives in the toolbox on that building site. Global installation is like installing a tool on your *personal tool belt* that you carry everywhere. You can use it on any building site, at any time, without needing to buy it again for each project. But you would not put every single tool on your belt — only the ones you use everywhere.

After a global install, the package's CLI command becomes available system-wide. For example, after `npm install -g nodemon`, you can type `nodemon` from any terminal window, in any folder.

## 1.3 When to Use Each

| Scenario                                                                                                    | Use                         | Reason                                                                        |
|-------------------------------------------------------------------------------------------------------------|-----------------------------|-------------------------------------------------------------------------------|
| A library your application code <code>require()</code> s (e.g., <code>express</code> , <code>axios</code> ) | Local                       | It needs to be in <code>node_modules</code> so Node.js can find it at runtime |
| A CLI tool you run from the terminal on many projects (e.g., <code>create-react-app</code> )                | Global                      | You need the command available everywhere, not tied to one project            |
|                                                                                                             | Local <i>dev dependency</i> |                                                                               |

| Scenario                                                                                        | Use | Reason                                                                                                                                                       |
|-------------------------------------------------------------------------------------------------|-----|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| A development tool used by all team members (e.g., <code>nodemon</code> , <code>eslint</code> ) |     | Keeps the tool version consistent across the team via <code>package.json</code> . A colleague with a different global version would get different behaviour. |

**Professional practice:** The Node.js community strongly prefers installing tools as *local dev dependencies* rather than globally — even tools like `nodemon` and `eslint`. This is because global installs are invisible to your project's `package.json`. A teammate who clones your project has no way to know which global tools are required. Local dev dependencies are self-documenting and reproducible.

## 1.4 Seeing Where Global Packages Are Installed

```
# See the folder where global packages live on your system
npm root -g

# List all globally installed packages
npm list -g --depth=0
```

### Output:

```
/home/alice/.nvm/versions/node/v20.11.0/lib/node_modules ← global root
├─ corepack@0.25.2
└─ npm@10.2.4
```

**NVM and global packages:** If you are using NVM (as recommended in Section 1), global packages are stored per Node.js version. This means if you switch from Node.js v20 to v18, your global packages from v20 are not available. Keep this in mind when working with multiple Node.js versions.

## 1.5 Removing Packages

To remove a package, use `npm uninstall`. NPM removes it from `node_modules` and automatically updates `package.json`.

```
# Remove a local package
npm uninstall chalk

# Remove a local dev dependency
npm uninstall nodemon

# Remove a global package
npm uninstall -g nodemon

# Short form
npm un chalk
```

## 2. Semantic Versioning

Every package in the NPM registry has a version number. These version numbers follow a universal standard called **Semantic Versioning**, often abbreviated as **SemVer**. Understanding it is essential because it controls which version of a package NPM installs and when it updates.

A semantic version number has three parts, separated by dots:



MAJOR

MINOR

PATCH

| Number | Name  | When it changes                                                                                           | Safe to auto-update?                         |
|--------|-------|-----------------------------------------------------------------------------------------------------------|----------------------------------------------|
| 4      | MAJOR | A <b>breaking change</b> — existing code may stop working.<br>Example: a function was renamed or removed. | No — requires manual review and code changes |
| 2      | MINOR |                                                                                                           |                                              |

| Number | Name  | When it changes                                                                               | Safe to auto-update?                          |
|--------|-------|-----------------------------------------------------------------------------------------------|-----------------------------------------------|
|        |       | A <b>new feature</b> was added in a backwards-compatible way. Your existing code still works. | Generally yes — new features, nothing removed |
| 1      | PATCH | A <b>bug fix</b> — no new features, no breaking changes. Just a correction.                   | Yes — always safe                             |

### Analogy

Think of an application you use daily — say, a note-taking app. A

### PATCH

update fixes a crash when you press save — invisible to you, always welcome. A

### MINOR

update adds a new dark mode — new capability, but your existing notes and workflows are untouched. A

### MAJOR

update restructures the entire app and moves all notes to a new format — your old notes might not work, and you need to spend time migrating. You would approach each type very differently.

## 2.1 Version Range Specifiers: `^`, `~`, and Exact

When NPM records a package version in `package.json`, it adds a prefix character that tells NPM how flexible it should be when looking for that package. There are three common forms:

### Exact Version — No prefix

```
"chalk": "5.3.0"
```

Install exactly version 5.3.0. No more, no less. NPM will never install a different version. Use this when a specific version is required and any change — even a patch — would be a problem. This is the strictest setting.

#### Tilde `~` — Patch-level flexibility

```
"chalk": "~5.3.0"
```

Install 5.3.0, and allow updates only within the PATCH number. NPM may install 5.3.1, 5.3.4, but will *never* install 5.4.0 or 5.5.0. The MINOR version is locked. Use `~` when you trust bug fixes but want to control which features are included.

#### Caret `^` — Minor-level flexibility (NPM default)

```
"chalk": "^5.3.0"
```

Install 5.3.0, and allow updates within the same MAJOR version. NPM may install 5.3.1, 5.4.0, 5.9.2, but will *never* install 6.0.0. The MAJOR version is locked. This is the default prefix NPM adds when you run `npm install`. It assumes that the package author follows SemVer correctly — that minor and patch updates will not break your code.

The table below summarises the allowed update range for each prefix when the installed version is `5.3.0`:

| Specifier in package.json              | Minimum | Allowed range | Will NOT install    |
|----------------------------------------|---------|---------------|---------------------|
| <code>"5.3.0"</code> (exact)           | 5.3.0   | 5.3.0 only    | 5.3.1, 5.4.0, 6.0.0 |
| <code>"~5.3.0"</code> (tilde)          | 5.3.0   | 5.3.0 – 5.3.x | 5.4.0, 6.0.0        |
| <code>"^5.3.0"</code> (caret, default) | 5.3.0   | 5.3.0 – 5.x.x | 6.0.0               |

**Special case — versions below 1.0.0:** For packages at version `0.x.x` (pre-release, not yet stable), `^` behaves more conservatively. `^0.3.0` only allows `0.3.x` updates — it treats MINOR as a breaking change, because pre-release software has no stability guarantees.

## 2.2 Practical: Installing Specific Versions

You can control exactly which version is installed by appending `@version` to the package name:

```
# Install the latest version (default)
npm install express

# Install an exact version
npm install express@4.18.2

# Install the latest within a major version
npm install express@4

# Install a version matching a range
npm install express@"^4.18.0"

# Install the most recent release tagged as "latest"
npm install express@latest
```

Let us install two versions of a package side-by-side to see how different version specifiers affect what ends up in `package.json`:

Terminal — run these commands one by one and observe `package.json` after each

```
# Start fresh
mkdir semver-demo && cd semver-demo && npm init -y

# Install with default caret (what npm install does)
npm install lodash@4.17.20

# Now install a different package at an exact version
npm install chalk@5.0.0
```

```
# And another with a tilde
npm install dayjs@~1.11.0
```

#### Output:

```
// The resulting "dependencies" section in package.json:
{
  "lodash": "^4.17.20",    ← caret added automatically by npm
  "chalk": "5.0.0",        ← exact, because we pinned it
  "dayjs": "~1.11.0"       ← tilde, because we specified it
}
```

**Remember:** The version in `package.json` is a *range declaration* (what you are willing to accept). The version in `package-lock.json` is the *exact version actually installed*. The two files serve different purposes and work together.

## 3. Updating Packages

Packages receive updates regularly — bug fixes, new features, security patches. Keeping your dependencies up to date is an ongoing maintenance responsibility for any real project. NPM provides two commands for this workflow: **check first, then update**.

### 3.1 `npm outdated` — Check Before You Update

Before blindly updating packages, you should first see *which* packages have updates available and *how significant* those updates are. `npm outdated` gives you a complete report:

```
npm outdated
```

#### Output:



| Package | Current | Wanted  | Latest  | Location            |
|---------|---------|---------|---------|---------------------|
| chalk   | 5.0.0   | 5.0.0   | 5.3.0   | node_modules/chalk  |
| dayjs   | 1.11.0  | 1.11.13 | 1.11.13 | node_modules/dayjs  |
| lodash  | 4.17.20 | 4.17.21 | 4.17.21 | node_modules/lodash |

The three columns are the most important part of this output:

| Column  | Meaning                                                                                                                                                                                                       |
|---------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Current | The version currently installed in <code>node_modules</code> on your machine right now.                                                                                                                       |
| Wanted  | The highest version allowed by your <i>current range specifier</i> in <code>package.json</code> . <code>npm update</code> will never go beyond this.                                                          |
| Latest  | The absolute latest version published to the NPM registry — regardless of your range. A red-coloured row means <b>Latest &gt; Wanted</b> , signalling a major version jump that requires manual intervention. |

Reading the output above:

- **chalk** — Current is 5.0.0. Wanted is also 5.0.0 (because we pinned it exactly with no `^` or `~`). Latest is 5.3.0, but NPM will not auto-update because of the exact pin. Manual action needed to upgrade.
- **dayjs** — Current 1.11.0 is behind Wanted 1.11.13. A safe update is available within the `~1.11.x` range.
- **lodash** — Current 4.17.20 is behind Wanted 4.17.21. One patch update available within `^4.x.x` range.

### 3.2 `npm update` — Apply Safe Updates

`npm update` updates all packages to their **Wanted** version — the highest version allowed by your current range specifiers. It will *never* exceed the range in `package.json`. This makes it safe to run.

```
# Update all packages to their allowed maximum (Wanted)
npm update

# Update a single specific package
```

```
npm update dayjs
```

```
# See what would be updated WITHOUT actually updating (dry run)
npm update --dry-run
```

#### Output:

```
changed 2 packages, and audited 3 packages in 0.6s
```

```
found 0 vulnerabilities
```

Running `npm outdated` again after the update shows dayjs and lodash are now up to date:

#### Output:

| Package | Current | Wanted | Latest | Location           |
|---------|---------|--------|--------|--------------------|
| chalk   | 5.0.0   | 5.0.0  | 5.3.0  | node_modules/chalk |

Only chalk remains — because its range is pinned exactly to `5.0.0`, `npm update` cannot change it. To update beyond a version range, you must manually edit `package.json` and then run `npm install`, or use `npm install chalk@latest` to explicitly request the latest version:

```
# Manually upgrade to a version outside your current range
npm install chalk@latest
```

```
# Or specify the exact new version
npm install chalk@5.3.0
```

**Warning — Major version updates require caution:** When a package releases a new MAJOR version (e.g., `express@4` → `express@5`), there are breaking changes. Before upgrading to a major version, always read the package's migration guide or changelog. Do not use `npm install pkg@latest` blindly on critical production dependencies — first check if the major version introduces breaking changes that affect your code.

## 4. Security Auditing: `npm audit`

The NPM registry maintains a public database of known security vulnerabilities in packages. When a vulnerability is discovered in a library, it is assigned a CVE (Common Vulnerabilities and Exposures) identifier and rated by severity. `npm audit` checks your installed packages against this database and reports any matches.

### Analogy

Think of `npm audit` as a building inspector checking your house against a list of known structural defects. The inspector does not fix anything — they produce a report listing which components have known issues and how serious each problem is. You then decide what to fix and when.

### 4.1 Running an Audit

```
npm audit
```

A clean project produces a reassuring result:

#### Output:

```
found 0 vulnerabilities
```

A project with vulnerabilities produces a detailed report. Here is a representative example of what it looks like:

#### Output:

```
# npm audit report

lodash <4.17.21
Severity: high
Prototype Pollution in lodash - https://github.com/advisories/GHSA-jf85-cpcp-j695
fix available via `npm audit fix`
node_modules/lodash

1 high severity vulnerability
```

To address all issues, run:  
`npm audit fix`

## 4.2 Understanding Severity Levels

| Severity        | Meaning                                                                                                   | Recommended action                                      |
|-----------------|-----------------------------------------------------------------------------------------------------------|---------------------------------------------------------|
| <b>Critical</b> | Can be exploited remotely without authentication. Immediate data loss or full system compromise possible. | Fix immediately. Stop and do not deploy until resolved. |
| <b>High</b>     | Significant impact — data exposure, denial of service, or privilege escalation.                           | Fix as soon as possible, within hours or days.          |
| <b>Moderate</b> | Limited impact, often requiring unusual conditions or limited attack surface.                             | Plan a fix within the current development cycle.        |
| <b>Low</b>      | Minimal real-world risk. May not be exploitable in practice.                                              | Fix in routine maintenance. Not urgent.                 |

## 4.3 `npm audit fix` — Automatically Resolving Vulnerabilities

`npm audit fix` attempts to automatically resolve vulnerabilities by updating packages to the lowest version that contains the security fix — staying within your declared version ranges.

```
# Fix all vulnerabilities that have a safe resolution within current ranges
npm audit fix

# Preview what would change WITHOUT making changes
npm audit fix --dry-run
```

Output:

```
changed 1 package, and audited 3 packages in 0.5s
```

```
found 0 vulnerabilities
```

## 4.4 When `npm audit fix` Is Not Enough

Sometimes a vulnerability can only be fixed by upgrading to a new MAJOR version of a package — a breaking change. In that case, `npm audit fix` will leave the vulnerability unresolved and report it. You must fix it manually.

To force the fix even if it involves a major version bump, use:

```
# Force major version upgrades to resolve vulnerabilities
# WARNING: This may introduce breaking changes – test thoroughly after
npm audit fix --force
```

**Warning:** `npm audit fix --force` can update packages across major version boundaries, introducing breaking changes into your code. Always run your tests after using this command, and review the changelog of any package that was major-version upgraded. Never use `--force` on a production deployment without thorough testing first.

## 4.5 Audit Report Formats

For CI/CD pipelines (automated build and deployment systems), you often need the audit results in a machine-readable format or want the build to fail if vulnerabilities are found:

```
# Output the audit report as JSON (useful for automated tools)
npm audit --json

# Only report vulnerabilities at or above a specific severity level
# Useful in CI: fail the pipeline only on high/critical issues
npm audit --audit-level=high
```

Output:

```
# If no HIGH or CRITICAL vulnerabilities found:
found 0 vulnerabilities

# Exit code is 0 – CI pipeline continues

# If HIGH/CRITICAL vulnerability found:
# Exit code is non-zero – CI pipeline fails and alerts the team
```

**Professional tip:** Add `npm audit --audit-level=high` to your CI/CD pipeline (GitHub Actions, Jenkins, GitLab CI, etc.). This automatically blocks deployments if a high-severity vulnerability is introduced — preventing insecure code from ever reaching production.

## 5. A Practical Dependency Maintenance Workflow

In real projects, dependency management is not a one-time task. It is done regularly throughout a project's lifetime. The following is a standard workflow that combines all the commands from this section.

Terminal – Full dependency health check and update sequence

```
# — Step 1: Check current state —————
# See which packages have available updates and how significant they are
npm outdated

# — Step 2: Check for known security vulnerabilities —————
npm audit

# — Step 3: Apply safe updates (within declared ranges) —————
# Always dry-run first to preview what will change
npm update --dry-run

# If the preview looks acceptable, apply
npm update

# — Step 4: Fix any security issues automatically —————
npm audit fix

# — Step 5: Check if anything remains —————
```

```

npm outdated
npm audit

# — Step 6: Commit the updated lock file —————
# Only package-lock.json changes – package.json ranges are unchanged
# git add package-lock.json
# git commit -m "chore: update dependencies"

```

After running this sequence, verify your application still works correctly by running your test suite ( `npm test` ) and starting the application. Never push dependency updates without verifying that the application still functions.

## Summary

| Concept / Command                              | Key Point                                                                                                   |
|------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| Local install ( <code>npm install</code> )     | Package goes into <code>./node_modules</code> . Scoped to this project. Use for application dependencies.   |
| Global install ( <code>npm install -g</code> ) | Package is available system-wide in the terminal. Use only for CLI tools you use across many projects.      |
| Exact version ( <code>"5.3.0"</code> )         | Install this exact version. No updates allowed.                                                             |
| Tilde ( <code>"~5.3.0"</code> )                | Allow patch updates only: 5.3.x                                                                             |
| Caret ( <code>"^5.3.0"</code> )                | Allow minor and patch updates: 5.x.x. NPM default.                                                          |
| <code>npm outdated</code>                      | Shows Current / Wanted / Latest versions. Identify what needs updating and how significant the updates are. |
| <code>npm update</code>                        | Updates packages to their Wanted version (within declared ranges). Safe to run.                             |

| Concept / Command                  | Key Point                                                                                                   |
|------------------------------------|-------------------------------------------------------------------------------------------------------------|
| <code>npm audit</code>             | Checks installed packages against the known vulnerability database. Reports severity levels.                |
| <code>npm audit fix</code>         | Automatically installs patched versions for vulnerabilities within safe ranges.                             |
| <code>npm audit fix --force</code> | Forces major version upgrades to resolve vulnerabilities. May introduce breaking changes — test thoroughly. |

## Interview Questions

1. What is the difference between `npm install lodash` and `npm install -g lodash` ?

*Expected answer:* Without `-g`, `lodash` is installed locally into the current project's `node_modules` and recorded in `package.json`. With `-g`, it is installed in a global system directory and available as a CLI command from anywhere — but not recorded in the project's `package.json`, making it invisible to other developers.

2. A package in `package.json` is listed as `"express": "^4.18.0"`. Which of these versions would NPM be allowed to install: 4.19.0, 4.18.5, 5.0.0, 3.9.0?

*Expected answer:* 4.19.0 ✓ (within major 4, higher minor) and 4.18.5 ✓ (within major 4, higher patch). Not 5.0.0 ✗ (different major — breaking changes) and not 3.9.0 ✗ (lower than the declared minimum).

3. What is the difference between `npm update` and `npm install pkg@latest` ?

*Expected answer:* `npm update` only updates packages to the maximum version allowed by their current range specifiers in `package.json` — it will never exceed those ranges. `npm install pkg@latest` explicitly installs the latest published version regardless of the current range, potentially jumping major versions and introducing breaking changes.



4. What do the three columns in `npm outdated` output represent?

*Expected answer:* **Current** is the version installed right now. **Wanted** is the highest version allowed by the range in `package.json` — what `npm update` will install. **Latest** is the absolute newest version on the NPM registry, regardless of your range.

5. You run `npm audit fix` but one critical vulnerability is still reported afterwards. Why might this happen?

*Expected answer:* The vulnerability's fix requires upgrading to a new MAJOR version of the package — a potentially breaking change. `npm audit fix` stays within safe version ranges and will not automatically apply breaking changes. The fix requires manually running `npm audit fix --force` (with caution) or manually updating the package version in `package.json` and running `npm install`, followed by testing.

6. Why is it generally better to install development tools like `nodemon` as local dev dependencies instead of globally?

*Expected answer:* A global install is not tracked in `package.json`, so other developers cloning the project have no way to know the tool is required. The specific version is also uncontrolled globally — team members may have different versions installed, causing inconsistent behaviour. A local dev dependency ensures everyone on the team uses the same version, and the tool is automatically installed when they run `npm install`.

7. What does `~1.2.3` mean as a version specifier, and how does it differ from `^1.2.3`?

*Expected answer:* `~1.2.3` allows only PATCH updates — versions 1.2.3 through 1.2.x. The MINOR version is locked. `^1.2.3` allows MINOR and PATCH updates — versions 1.2.3 through 1.x.x. The MAJOR version is locked. Caret is more permissive; tilde is more conservative.

## Practical Assignment

**Title:** Dependency Health Check — Version Control & Audit Workflow

**Objective:** Practice reading version specifiers, identifying outdated packages, and responding to a security audit.

### Setup: Create a project with intentionally mixed version specifiers

1. Create a new project folder `dep-health` and initialise it with `npm init -y`.
2. Install the following packages at the specific versions and specifier types shown below by manually editing the `"dependencies"` section of `package.json`, then running `npm install`:

```
"dependencies": {  
  "lodash": "4.17.20",  
  "dayjs": "~1.11.0",  
  "chalk": "^5.0.0",  
  "axios": "0.21.1"  
}
```

3. Run `npm outdated` and for each package, answer in a comment in your code:
  - What is the Current version?
  - What is the Wanted version?
  - What is the Latest version?
  - Which specifier rule is preventing an update (if any)?
4. Run `npm update` and then run `npm outdated` again. Which packages were updated? Which were not, and why?
5. Run `npm audit`. Note all vulnerabilities found. Run `npm audit fix` and check again. Document any that could not be fixed automatically and explain why.
6. Create an `index.js` that `require()`s all four packages and logs the current version of each using their respective APIs or the `require('package/package.json').version` pattern.

Expected final state of `npm outdated` (after all fixes):

#### Output:

```
# Ideally: empty output (no outdated packages)  
# Or only entries where Latest > Wanted due to exact pinning or major version lock
```

**Bonus challenge:** Add an NPM script called `"health"` that runs `npm outdated && npm audit` in sequence — a one-command project health check.

---

Module 4: Node.js & Express · Section 2 — Package Operations · Next: **Section 2 (continued) — Workspaces and Alternative Tools**  
(Yarn, pnpm)

